Lenguajes de Programación

Clase 8: Principios Fundamentales de la POO

Docentes:

- Ing. Sebastián Balestrini
- Ing. Gustavo Distéfano

🎯 Objetivos

- Comprender los principios de **encapsulamiento**, **herencia** y **polimorfismo**.
- Analizar cómo estos principios ayudan a organizar y reutilizar código.
- Aprender a representar relaciones entre clases mediante un **diagrama UML básico**.
- Ver ejemplos básicos de estos conceptos en **C++** y **Java**.

🧰 Herramientas

- Lucidchart
- Online GDB

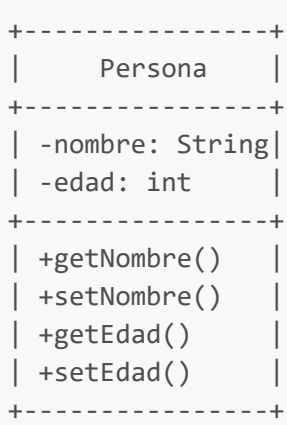
💡 Principios de la Programación Orientada a Objetos

> "La POO organiza mejor el código mediante principios clave que facilitan reutilización, mantenimiento y legibilidad."

🔓 Encapsulamiento

- Ocultar los detalles internos de una clase y exponer solo lo necesario.
- Protege los atributos usando **modificadores de acceso**.
- Permite mantener control sobre los datos.

Ejemplo UML (Encapsulamiento)



Ejemplo en C++

```cpp
#include <iostream>
using namespace std;

class Persona {
private:
    string nombre;
    int edad;

public:
    void setNombre(string n) { nombre = n; }
    string getNombre() { return nombre; }

    void setEdad(int e) { edad = e; }
    int getEdad() { return edad; }
};

int main() {
    Persona p;
    p.setNombre("Juan");
    p.setEdad(25);
    cout << p.getNombre() << " tiene " << p.getEdad() << " años." << endl;
    return 0;
}
```

Ejemplo en Java

```java
public class Persona {
    private String nombre;
    private int edad;

    public void setNombre(String n) { nombre = n; }
    public String getNombre() { return nombre; }

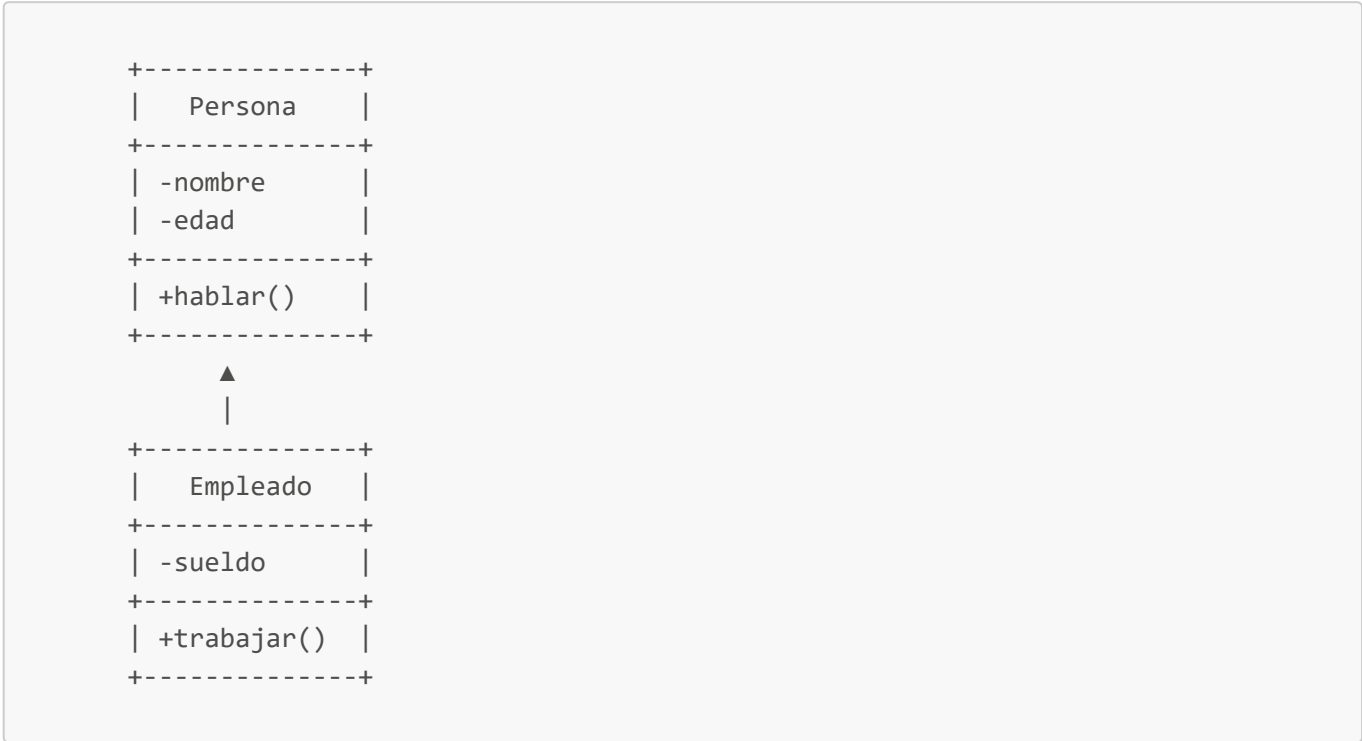
    public void setEdad(int e) { edad = e; }
    public int getEdad() { return edad; }

    public static void main(String[] args) {
        Persona p = new Persona();
        p.setNombre("Juan");
        p.setEdad(25);
        System.out.println(p.getNombre() + " tiene " + p.getEdad() + " años.");
    }
}
```

Herencia

- Una clase hija **hereda atributos y métodos** de una clase padre.
- Permite **reutilizar código** y extender comportamientos.
- La relación entre clases es de tipo **“es-un”**.

Ejemplo UML (Herencia)



Ejemplo en C++

```cpp
#include <iostream>
using namespace std;

class Persona {
public:
    void saludar() { cout << "Hola, soy una persona." << endl; }
};

class Empleado : public Persona {
public:
    void trabajar() { cout << "Estoy trabajando." << endl; }
};

int main() {
    Empleado e;
    e.saludar();
    e.trabajar();
    return 0;
}
```

Ejemplo en Java

```java
class Persona {
    void saludar() {
        System.out.println("Hola, soy una persona.");
    }
}

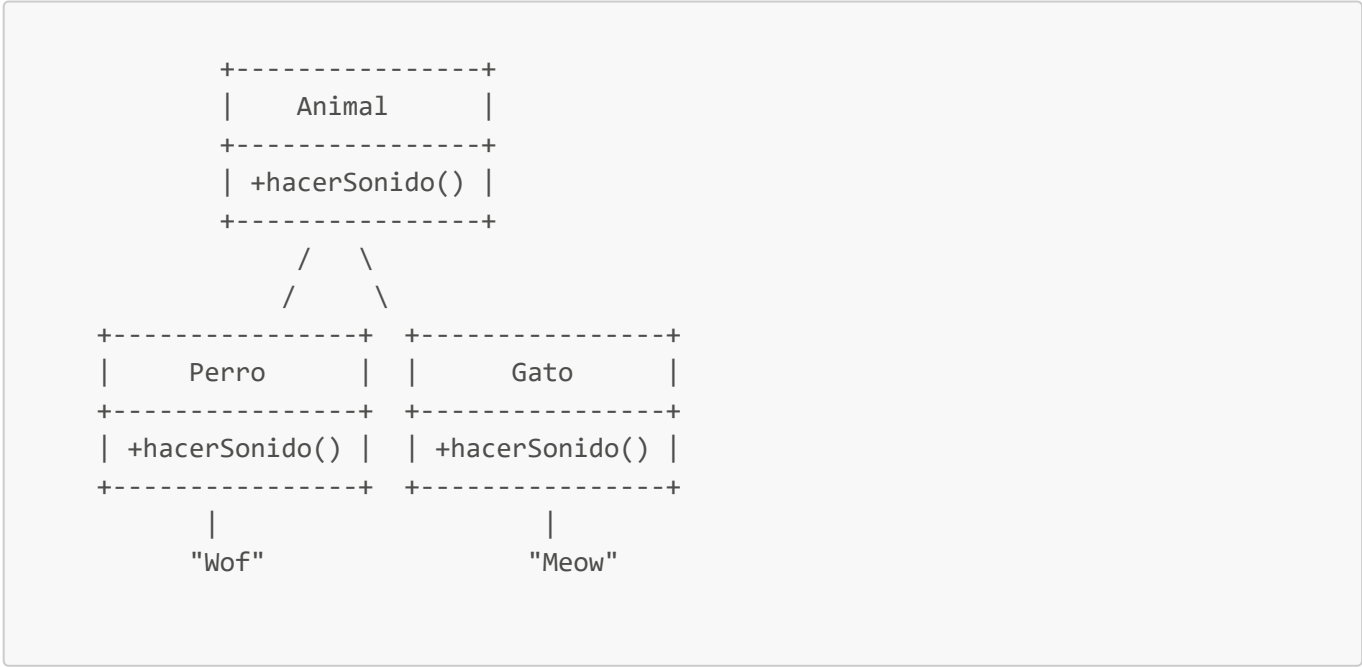
class Empleado extends Persona {
    void trabajar() {
        System.out.println("Estoy trabajando.");
    }
}

public class Main {
    public static void main(String[] args) {
        Empleado e = new Empleado();
        e.saludar();
        e.trabajar();
    }
}
```

Polimorfismo

- Un mismo método se comporta diferente según el objeto que lo use.
- Permite **sobrescribir métodos** en clases hijas.

Ejemplo UML (Polimorfismo)



¿Qué es el Polimorfismo?

> Es la capacidad que tienen los objetos de una clase en ofrecer respuestas distintas e independientes frente a un mismo mensaje o método.

- Por ejemplo, tanto Perro como Gato heredan de Animal, pero cada uno implementa su propio comportamiento del método hacerSonido().

Ejemplo en C++

```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void hacerSonido() { cout << "Sonido genérico" << endl; }
};

class Perro : public Animal {
public:
    void hacerSonido() override { cout << "Guau!" << endl; }
};

class Gato : public Animal {
public:
    void hacerSonido() override { cout << "Miau!" << endl; }
};

int main() {
    Animal* a1 = new Perro();
    Animal* a2 = new Gato();
    a1->hacerSonido();
    a2->hacerSonido();
    delete a1;
    delete a2;
    return 0;
}
```

Ejemplo en Java

```java
class Animal {
    void hacerSonido() {
        System.out.println("Sonido genérico");
    }
}

class Perro extends Animal {
    void hacerSonido() {
        System.out.println("Guau!");
    }
}
```

```
class Gato extends Animal {
    void hacerSonido() {
        System.out.println("Miau!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a1 = new Perro();
        Animal a2 = new Gato();
        a1.hacerSonido();
        a2.hacerSonido();
    }
}
```

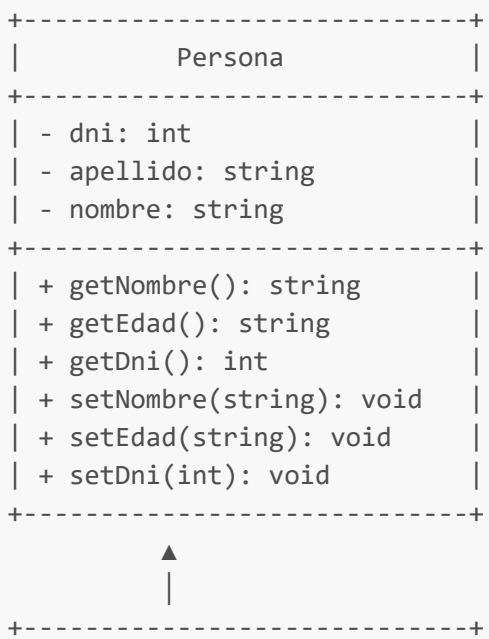
☑ Ventajas de la POO

- Reutilización de atributos y métodos.
- Abstracción.
- Modularidad.
- Mayor flexibilidad y escalabilidad.
- Permite crear sistemas complejos.
- Agiliza el desarrollo de software.

UML

- Visualiza clases, atributos, métodos y relaciones.
- **Flecha con triángulo vacío = herencia.**

Ejemplo:



```
              |         Estudiante          |
              +-----------------------------+
              | - legajo: string            |
              +-----------------------------+
              | + getLegajo(): string       |
              | + setLegajo(string): void   |
              +-----------------------------+
```

RESUMEN

Encapsulamiento

- Oculta atributos y expone solo métodos de acceso.
- Mejora la organización y seguridad del código.

Herencia

- Una clase hija reutiliza atributos y métodos de una clase padre.
- Evita duplicar código y permite extender funcionalidades.

Polimorfismo

- Un mismo método se comporta diferente según el objeto.

UML

- Visualiza clases, atributos, métodos y relaciones.
- Flecha con triángulo vacío = herencia.
